AI23531 – DEEP LEARNING

EX 1 THREE-LAYER NEURAL NETWORK FOR HANDWRITTEN DIGIT RECOGNITION (MNIST)

AIM

To design and train a three-layer neural network (input, hidden, output) using backpropagation with suitable activation and loss functions, and apply it to recognize handwritten digits from the MNIST dataset.

PROCEDURE

1. Import NumPy, Matplotlib, TensorFlow and Keras.
2. Load the MNIST dataset and normalise pixel values to the range 0–1.
3. Flatten each 28×28 image into a 784-element vector.
4. Build the network: Dense(128, ReLU) → Dropout(0.2) → Dense(10, Softmax).
5. Compile with the Adam optimizer and sparse categorical cross-entropy loss.
6. Train for 5 epochs (batch size 32) with a 20% validation split.
7. Evaluate on the test set and display predictions for sample test images.

ALGORITHM

1. Start.
2. Load MNIST training and test sets; scale inputs by 1/255.
3. Reshape the images to 784-dimensional vectors.
4. Initialise the layers: 784 inputs → 128 hidden neurons (ReLU) → 10 outputs (Softmax).
5. For each epoch and each mini-batch: forward pass, compute cross-entropy loss, backpropagate gradients, update weights with Adam.
6. Record training and validation accuracy and loss after every epoch.
7. Evaluate on the test data and report test accuracy.
8. Predict digits for the first five test images using argmax of the output; display them with predicted and actual labels.
9. Stop.

PROGRAM

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist

(x_train, y_train), (x_test, y_test) = mnist.load_data()

x_train = x_train.astype('float32') / 255
x_test = x_test.astype('float32') / 255

x_train = x_train.reshape((x_train.shape[0],28*28))
```

```
model = models.Sequential()

model.add(layers.Dense(128,
activation='relu',
input_shape=(784,)))

activation='relu'

model.add(layers.Dropout(0.2))

model.add(layers.Dense(10,
activation='softmax'))

model.compile(
optimizer='adam',
loss='sparse_categorical_crossentropy',
metrics=['accuracy']
)

model.fit(
x_train,
y_train,
epochs=5,
batch_size=32,
validation_split=0.2
)

x_test = x_test.reshape((x_test.shape[0], 28 * 28))

test_loss, test_acc = model.evaluate(x_test, y_test)
print(f"Test Accuracy: {test_acc:.4f}")

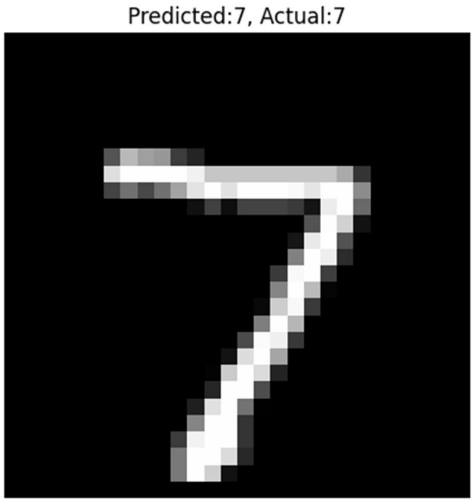
predictions = model.predict(x_test)

for i in range(5):
    plt.imshow(x_test[i].reshape(28,28), cmap='gray')
    plt.title(
        f"Predicted:{np.argmax(predictions[i])}, Actual:{y_test[i]}"
    )
    plt.axis('off')
    plt.show()
```

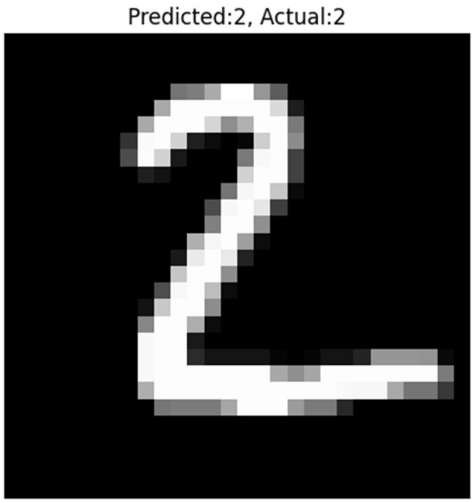
OUTPUT

```
Epoch 1/5
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9023 - loss: 0.3325 - val_accuracy: 0.9540 - val_loss
: 0.1652
Epoch 2/5
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.9502 - loss: 0.1678 - val_accuracy: 0.9638 - val_los
s: 0.1244
Epoch 3/5
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.9635 - loss: 0.1225 - val_accuracy: 0.9666 - val_loss
: 0.1106
Epoch 4/5
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.9700 - loss: 0.1009 - val_accuracy: 0.9715 - val_loss
: 0.0999
Epoch 5/5
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9734 - loss: 0.0856 - val_accuracy: 0.9717 - val_loss
: 0.0970
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9727 - loss: 0.0905
Test Accuracy: 0.9727
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
```

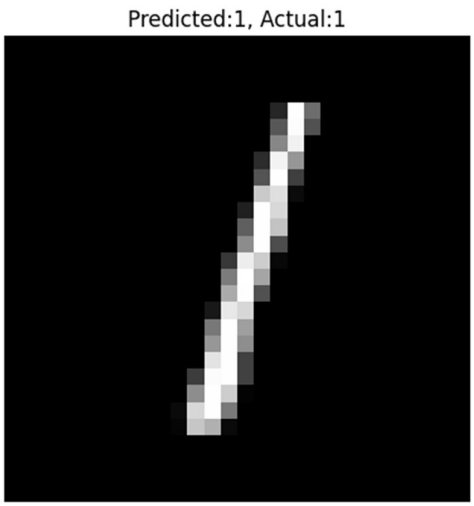
Console output



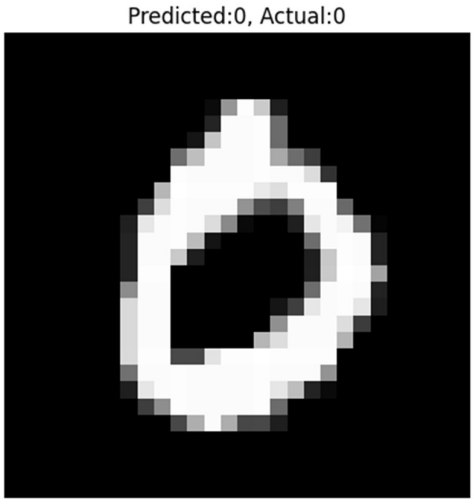
Predicted vs actual digit (sample test image 1)



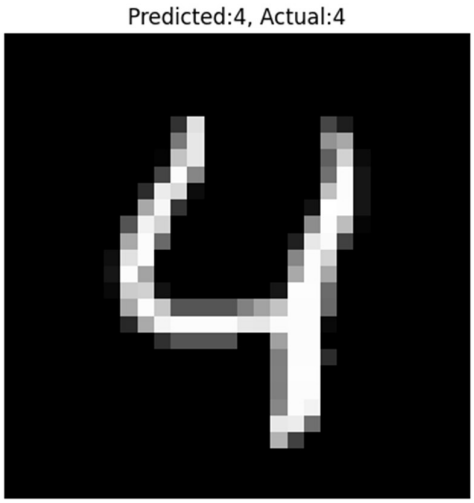
Predicted vs actual digit (sample test image 2)



Predicted vs actual digit (sample test image 3)



Predicted vs actual digit (sample test image 4)



Predicted vs actual digit (sample test image 5)

RESULT

The network reached a test accuracy of 97.27% on MNIST and correctly recognized the sample handwritten digits.